Spectral Analysis of Traffic Accidents in New York's Capital Region

by

Michael Barr

A Thesis

Submitted to the University at Albany, State University of New York

in Partial Fulfillment of the Requirements for the Degree of

Master of Science, Biostatistics

College of Integrated Health Sciences

Department of Biostatistics

May 2026

Abstract

In this paper we estimate the spectral density of traffic accident events in the Capital Distict, NY area using a band-pass filter known as the Kolmogorov-Zurbenko Fourier Transform (KZFT). The source data is provided by Moosavi, et al. (2019) and originally captured from various public entities and sensors in the road network. Spectral density estimation with KZFT suppresses noise to reveal a greater number of constituent frequencies in the signal compared to standard fourier analysis. We are able to produce more accurate estimated energies, in particular for the many lesser harmonics of the fundamental weekly frequency detected in accident patterns. Signal reconstruction based on KZFT produces the approximate weekly pattern of this noisy signal which is not governed by any underlying physical processes transmitting sinusoidal waveform energy. KZFT and spectral analysis methods in general are more typically applied to signals which are highly determined by physical forces and processes - objects that spin, oscillate, vibrate, orbit, and so on. Our novel application of these methods to population patterns measured by highly random loss events demonstrates their potential for greater adoption among researchers working with signals from varied fields.

Introduction

High frequency, remotely-sensed data is increasingly available to researchers in industry, government, and academia. Sensors have long been utilized to measure and monitor a wide variety of phenomena and conditions in environmental (air quality, ocean tides, atmospheric pressure, seismic activity), industrial or mechanical equipment (heat, vibration, alignment, etc. within motors, gearboxes, pumps, and various rotating equipment), built infrastructure (e.g. vibration sensors for structural health monitoring of bridges), and many other areas. Technological innovation on multiple fronts in recent years has reduced the size and cost of sensor hardware (Microsoft 2019)[1], increased options for connectivity (e.g. with short-range/bluetooth connectivity to smartphones, local wireless networks, and private 5G wireless networks in some commercial settings), and exponentially reduced the cost of physical data storage (McCallum 2023). These developments have fostered a massive expansion of "smart" product offerings for consumer and commercial applications including wearable biometric monitoring devices, home systems monitoring (temperature, moisture/leak, and arc fault detection), and automotive telematics devices measuring aspects of vehicle usage and driving behavior (acceleration/deceleration, speed, location, time of day). Many of these and other applications are linked to insurance products with the purpose of measuring, pricing for, and studying the effects of interventions designed to reduce the risk of loss events affecting health or property (Gundla 2025).

The increased deployment of sensors and the data streams they produce demands practitioners who are equipped with the statistical methodologies suited for their analysis. A series of measurements taken sequentially over time - whether at regular or irregularly spaced intervals - is a time series. Methods for the analysis of time series have in our view received less attention than warranted. This could be due in part to the concurrent development and focus on new machine learning techniques for regression analysis and advanced prediction systems in the context of data that is tabular, un-ordered, and rich with covariates. To the extent that methods for time series are given attention it is usually limited to time-domain

methods - the auto-regressive model and its extensions.

The task of this paper is to demonstrate the relevance of "the other half" of time series analysis - spectral analysis - for data we may not immediately see as calling for it. Part I of this paper provides a brief motivation for spectral analysis with an overview of key concepts building to the frequency domain and the (Fast) Fourier Transform. From this groundwork we proceed to discuss the Kolmogorov-Zurbenko (KZ) Filter, a method for extracting frequencies from signals in the time domain. In Part II we discuss the extension of the KZ filter to spectral density analysis with the Kolmogorov-Zurbenko Fourier Transform (KZFT) band-pass filter. Part III demonstrates application of KZFT to a traffic accident event data set sourced from publicly available, streaming APIs which monitor and report traffic incidents in real-time. KZFT reveals a fundamental weekly frequency with harmonics underlying the signal we construct from this event data. We use the results to reconstruct the pattern of the signal and compare the result to the a "null" model obtained in the time domain.

Part I: Background

Temporal and Frequency Domains

Assume a random variable X_t varying over time, the result of some data generating process involving both deterministic and stochastic components. A time-series is a sequence of measurements $\{X(t) : t \in 0, 1, 2, ..., n\}$ taken of X_t, recorded successively, indexed and ordered in time. X_t may be multivariate - that is, each measurement $X(t)$ a vector of values. For simplicity we will deal with the univariate case - we assume X_t is scalar-valued. We will also assume X_t is real-valued though this is not strictly necessary. A time series is the result of a data generating process along with some repeatable means of obtaining and recording measurements - which may be imprecise - of the observable value at points in time. We will refer to the means of obtaining measurements generally as a *sensor*. A sensor may refer to

anything from a piece of electronic hardware recording and digitally transmitting readings to a human recording measurements with the aid of a primitive mechanical instrument. A series of measurements obtained of X_t is often referred to as the "raw signal" or simply the *signal*.

Methods for time-series analysis may seek to describe the deterministic and stochastic components embedded in X_t, often by estimating a model from the signal. Analysis methods can be classed into two broad categories. *Time-domain* methods work with the measurements or *total energy* of the observed signal in the time domain we naturally perceive and record it. This includes any approach which views the data temporally, whether calculating a simple moving average, estimating an AR(p) model and its variations, or training a Long Short-Term Memory network. Such methods are often best suited for signals which do not contain many underlying cyclical components - e.g. when X_t is highly determined by the most recent lagged values $X_{t-1}, X_{t-2}, ...$ A brownian motion in its discrete form - also known as a random walk - is the archetypical example.

When the data generating process underlying a signal is more complex - that is, when it is governed by multiple cyclical components - then analysis in the time domain can quickly become intractable. *Frequency-domain* methods begin with some means of transforming a time-indexed signal into a spectrum of estimated energies in order that we may better detect and describe the cyclical components ("constituents" or "waveforms") embedded in a complex and potentially noisy signal.

Figure 1 illustrates the issue with a simulated signal (top) generated by three cyclical constituents (together the "systematic component") and a white noise (the stochastic component). The expected value of the generating function is shown in total (second panel) and decomposed into its three constituents (third). The stochastic component is also shown (bottom).

Even a relatively simple signal like above poses a challenge for analysis in the time domain. Consider the de-noised signal - after removing the stochastic component it remains

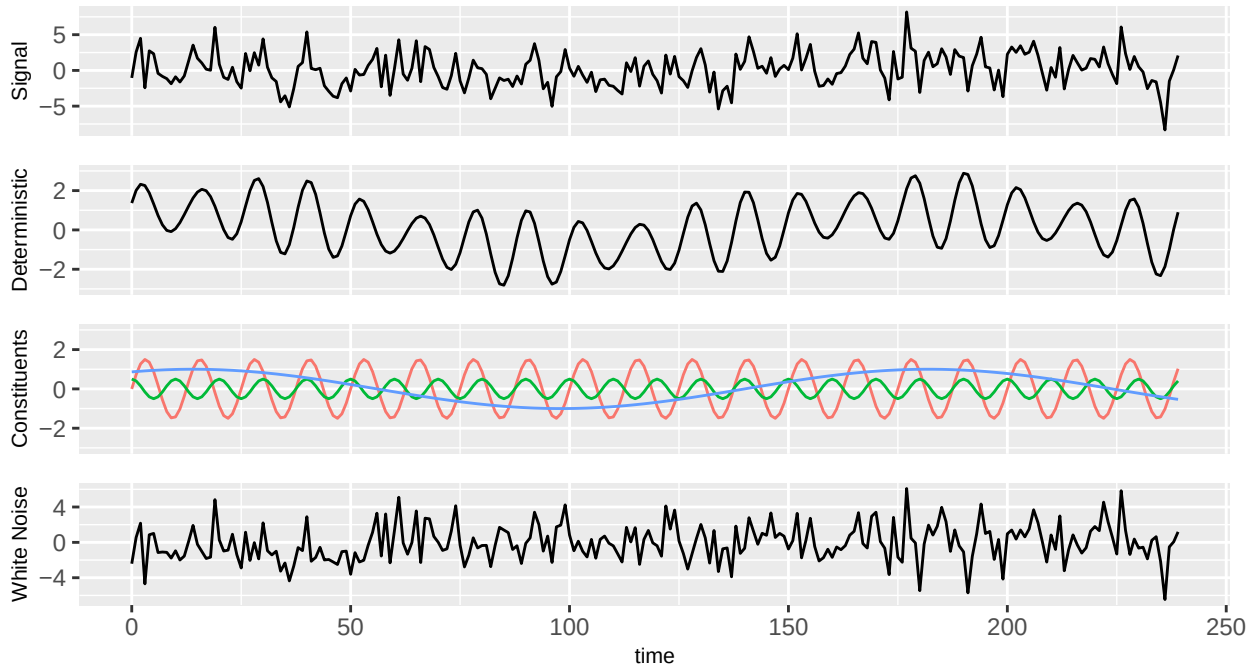

Figure 1: $X(t) = 1.5\sin(\frac{2\pi}{12.5}) + .5\sin(\frac{2\pi}{10}) + \sin(\frac{2\pi}{168}) + N(0, \sigma = 2)$

not obvious what underlies the pattern we observe in time. We can detect the dominant frequency at .08 (period = 12.5), but the relative magnitude at this frequency varies as it is modulated by phase-alignment with the waveform at frequency .10 every 4th period. The absolute magnitude is also seen to cycle more slowly with the third constituent. None of this is apparent in the raw signal, however, which appears very messy in the time domain despite a relatively high signal-to-noise ratio of .446. Though an AR(2) model can be used to perfectly describe a single sine wave without noise, the order required becomes prohibitively high as a signal becomes more complex. Analysis of partial autocorrelation is not a help because - like the AR(p) model - it is based in the assumption that the greatest predictive information is found in the nearest lagged values. In fact the predictive information for the next value $X(t)$ is found in the lags corresponding to the cycle times (multiples of $\frac{1}{f_\nu}$) of each waveform embedded in the signal - e.g. $X(t-10)$, $X(t-20)$,...,$X(t-\frac{c}{0.1})$ for a waveform present at frequency $f_{.1}$. When random variation is present in the signal, the auto-regressive model is misspecified and estimates obtained from this approach are biased.

This signal can be neatly summarized within the frequency domain. A periodogram

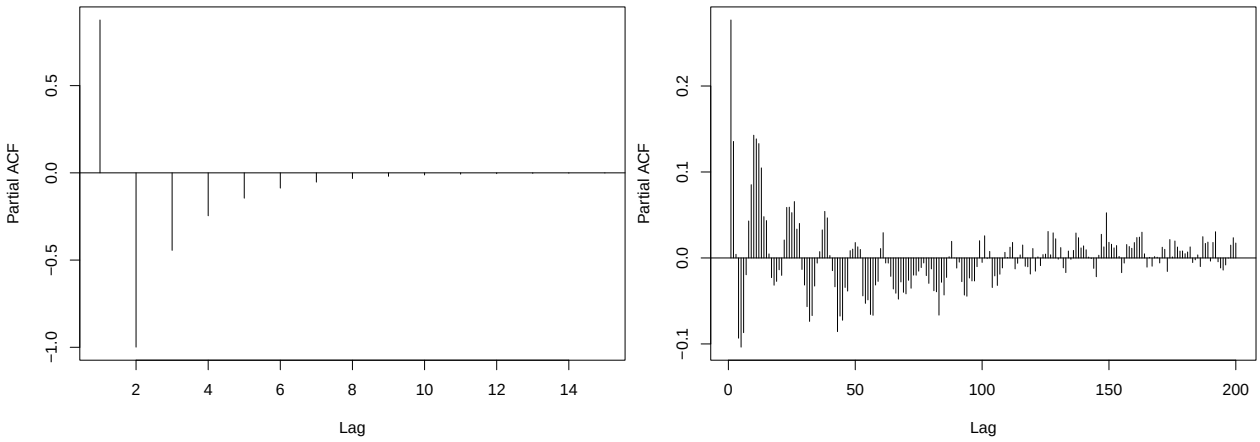

Figure 2: (left) Using PACF to detect the order, or key lags, of an AR(p) model for a single sine wave at $f_{\frac{1}{12}}$ without noise; (right) PACF applied to the simulated X_t with SNR=.446

represents the actual or, more often, estimated spectrum of a signal. It is often presented on the power (= energy^2) scale, but may be shown on energy or information scales, may be log-transformed, and may or may not be normalized for the number of samples in the signal. When presented on the power scale as is convention we may refer to it as the power spectral density (PSD). Regardless of these scaling choices, the estimates describe assumed sinusoidal waveforms oscillating at each observable "frequency bin" or Fourier frequency from f_0 (the "DC component" or mean power level) up to the Nyquist limit $\frac{f_S}{2}$ (half the sampling rate of the signal). The power spectrum behind a signal may in some cases be known, but usually for a signal found "in nature" and measured in a research context it must be estimated in the presence of noise. If we can identify the constituent frequencies which determine a signal, estimate their respective energy levels and phase or offset at $t = 0$, then we can describe and reconstruct the unobservable systematic component behind the signal. We can also use this information to forecast future values of X_t yet to be realized, assuming the process is stationary over time. Figure 3 shows the (known) PSD of the deterministic component generating the above signal.

The basic task of spectral analysis is to decompose a potentially complex and noisy signal into a set of non-random constituent waveforms and residual variance or unexplained noise by estimating the distribution of power across a spectrum of observable frequencies.

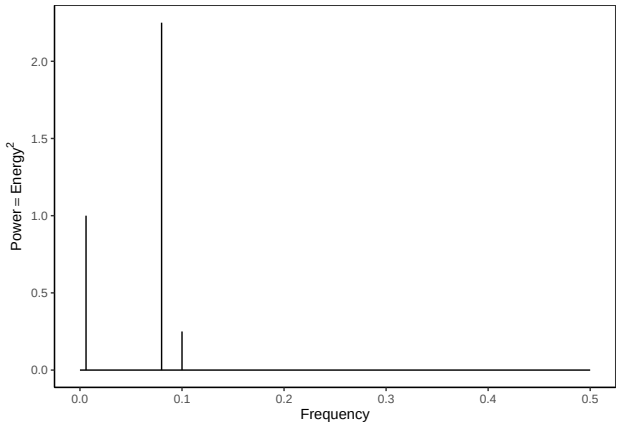

Figure 3: Theoretical periodogram (systematic component) of simulated signal

The periodogram is classically defined as $\frac{1}{N}|FFT(X_t)|^2$ based on the Fast Fourier Transform of X_t. The inverse FFT applied to the transformed signal returns the original signal. The transform then can be thought of as a saturated model - it produces a function over the frequency domain with as many parameters as there are samples in the time domain. As an estimator for the PSD this approach is asymptotically unbiased but also inconsistent and high variance - the result is often a highly noisy power spectrum which does not become smoother with greater samples. We desire an estimator θ which is not necessarily unbiased but instead optimizes a trade-off between bias and variance to minimize mean square error of the estimator.

$$MSE(\theta) = bias_\theta^2 + var(\theta)$$

We now turn to reviewing the foundational method of spectral density analysis known as the fourier transform.

Fourier Transform

It is well known that any function $f(t)$ defined over a self-symmetric domain can be decomposed into even and odd parts, and therefore uniquely expressed as the sum of an even and odd function.

$$f(t) = f_e(t) + f_o(t)$$

$cos(t)$ is an even function - i.e. it is symmetrical about the y-axis ($cos(-x) = cos(x)$), and $sin(t)$ is an odd function ($sin(-t) = -sin(t)$). A sine function can of course be written as a cosine with phase shift $\phi = \frac{\pi}{2}$. In general, a sine wave with phase offset can be expressed as a linear combination of a sine and cosine each without phase: $a\sin(t+\phi) = a\cos(\phi)\sin(t) + a\sin(\phi)\cos(t)$. Put another way, one sine and one cosine function with identical period and 0 phase together form a complete (and othogonal) basis set for expressing an arbitrary sine function of the same period. This fact can be generalized to the complex case:

$$ja\sin(t+\phi) = z\Big(\cos(t) + j\sin(t)\Big)$$

where z is a complex-valued constant containing both a and ϕ. We can allow that the imaginary part of the left term may be 0 - so we can express a sinusoid on the real plain with non-zero phase as the scaled sum of complex-valued sine and cosine functions without phase.

Finally, by Euler's identity we know that $e^{jt} = \cos(t) + j\sin(t)$, so the expression above can be restated as

$$ja\sin(t+\phi) = ze^{jt}$$

The Fourier Transform allows us to express any function $f(t)$ over a time domain as the integral of sine functions $F(\nu)$ spanning the frequency domain, each of them parameterized by some amplitude and phase.

$$f(t) = \frac{1}{2\pi}\int_{-\infty}^{\infty} F(\nu)e^{j\nu(t)}d\nu$$
$$F(\nu) = A(\nu)e^{j\phi(\nu)}$$

Each $F(\nu)$ is a complex exponential at a given frequency. The set of all $F(\nu)$ forms a complete and orthogonal basis in the frequency dimension - they are pairwise independent and can be linearly combined to represent an arbitrary function $f(t)$.

Though we experience it every day, typically we cannot work with data in continuous

time. Either our sensors will have a sampling rate or an analogue (continuous) signal must be digitized (sampled) for storage. So all signals will be defined by a sampling rate $f_S = \frac{1}{dt}$ and our methods will work in discrete time. The Discrete Fourier Transform (DFT) analogue to the above is more relevant in practice for regularly spaced signal data. The DFT is defined as

$$F(k) = \sum_{t=0}^{N-1} X_t e^{-j2\pi k \frac{t}{N}}$$

with k the frequency index, t the time index, and N the number of samples in the signal. The frequency index k corresponds to physical frequency $\frac{k}{N} f_S$.

Spectral density analysis was enabled in principle by the development of what came to be known as the Fourier Transform based on work published by Jean-Baptiste Joseph Fourier in the 1820s in his study of heat transfer (Valentinuzzi, 2016).[2] But it was its implementation via the Cooley-Tukey algorithm - the Fast Fourier Transform (FFT) - in 1965 that made spectral analysis feasible in the modern setting. Most algorithms today are implemented using FFT rather than direct calculation from the definition due to the difference in computational complexity. Direct calculation of the DFT from its definition requires $O(N^2)$ operations for the full spectrum, while FFT reduces this to $O(N \log N)$. This means for a series of 1,000 samples the DFT performs 1,000,000 operations while FFT requires only 6,908. The performance gain comes with a key limitation - missing values cannot be present in the input signal due to the recursive approach of the FFT algorithm.

FFT is unreliable as an estimator for the PSD, in particular when signal-to-noise is low and waveforms are located near one another in the spectrum. Common approaches to PSD estimation will build from FFT a consistent estimator either by smoothing the spectral output or through different strategies of repeated application to subsets of the total signal. Variations of the latter include breaking up the original signal into equal size blocks which may or may not overlap and applying FFT separately to each before combining results (Short-Time Fourier Transform), and applying FFT repeatedly within a sliding window over the data (Sliding Window Fourier Transform). In place of a rectangular window, tapering

windows are often preferred to reduce signal side lobes, which has the effect of reducing sharp discontinuities at the edges of a signal which contribute to high frequency noise. Windowing sacrifices frequency resolution in order to obtain the variance reduction produced by averaging of multiple estimates, and in particular when non-rectangular windows are used (Harris, 1978).

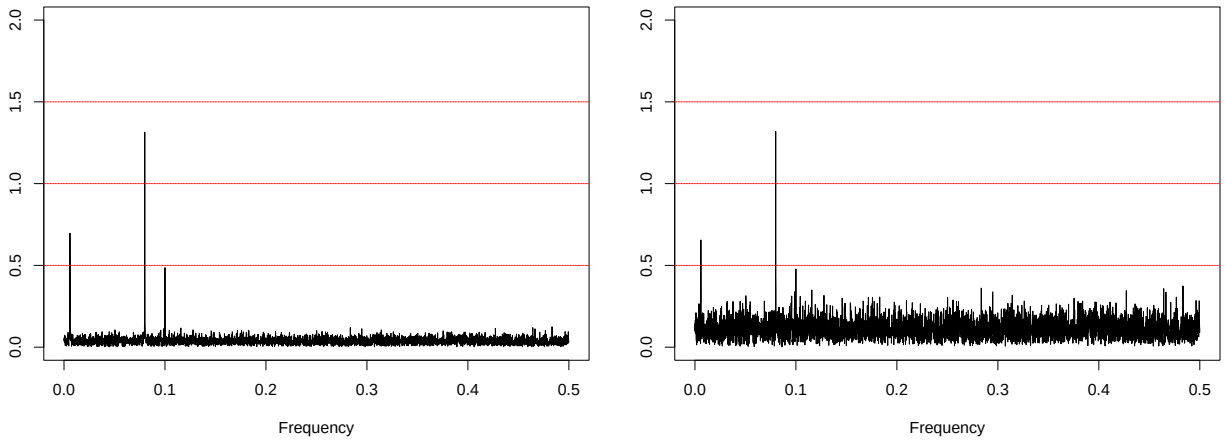

Figure 4: Periodogram from FFT (normalized for sample size and expressed in energy/amplitude) on simulated data with SNR=.446 (left) and with noise amplified to SNR=.05 (right). The estimates are poor at either noise level. The $f_{\frac{1}{12.5}}$ (amplitude = .5) constituent is nearly buried in the higher noise.

Waveform Shapes

Although FFT uses a basis of sinusoidal waveforms to decompose a signal in the frequency domain, the waveforms actually present need not be sinusoidal for the result to be meaningful. Sinusoids may be combined to approximate waveforms of arbitrary shape. Some examples which arise include rectangular, triangular, and sawtooth patterns in time. The spectra produced by FFT in such cases are characterized by energies located at multiples or *harmonics* of the fundamental frequency at which a waveform is located. So long as cyclic behavior is present, spectral analysis with FFT and its variations is viable.

We return now to the temporal domain to introduce the KZ filter, upon which the KZFT method for spectral density estimation is based.

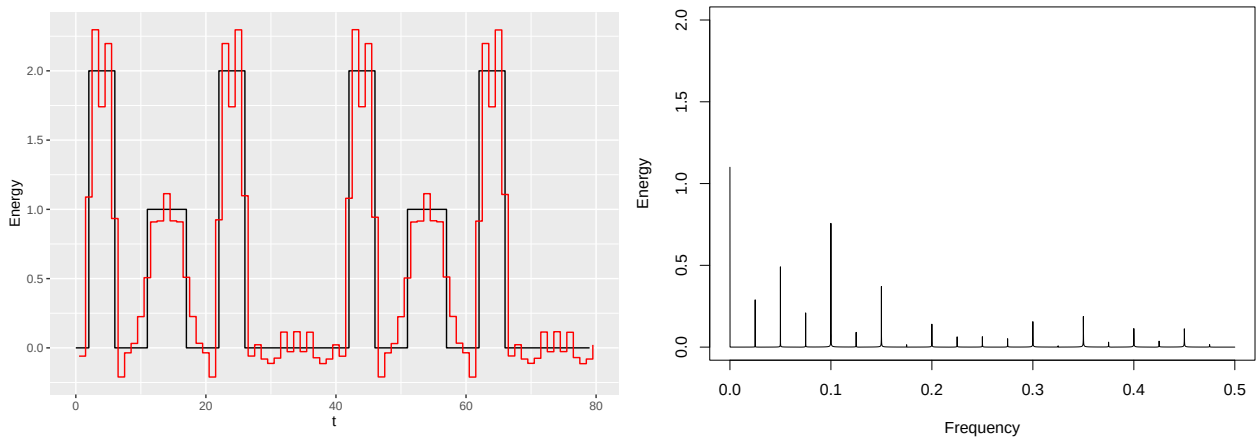

Figure 5: Segment from a composite signal of 2 rectangular waveforms with periods .05 and .025, and its reconstruction (left); the energy density estimated by FFT (right). The reconstruction is performed using the 10 harmonics with highest energy and the f_0 component.

Kolmogorov-Zurbenko Filter

Often the analysis of a signal exhibiting a high degree of noise begins with some simple exploratory techniques. A signal may be "smoothed" with some parametric or nonparametric approach to reduce the presence of noise and extract cyclical patterns. The KZ is a relatively simple non-parametric approach for this task, defined as a sliding window arithmetic mean applied k-times iteratively to a signal. Let $\{X(t) : t = 0, \pm 1, \pm 2, ...\}$ be a real-valued, univariate time series at any measurement interval. Then for one iteration ($k = 1$), the KZ filter applied at observation $X(t)$ is given by:

$$KZ_{m,k=1}\Big(X(t)\Big) = \sum_{s=-(m-1)/2}^{(m-1)/2} X(t+s) \times \frac{1}{m}$$

where m odd is the bandwidth of the chosen window. With $k > 1$ iterations the smoothing of the series is repeated on the result of the preceding iteration. The KZ filter can then be defined generally as

$$KZ_{m,k}\Big(X(t)\Big) = \sum_{s=-k(m-1)/2}^{k(m-1)/2} X(t+s) \times a_s^{m,k}$$

where coefficients $a_s^{m,k}$ are the observation weights determined by polynomial coefficients for the window size, given by

$$a_s^{m,k} = \frac{c_s^{k,m}}{m^k} = \frac{1}{m^k} \sum_{r=0}^{k(m-1)} z^r c_{r-k(m-1)/2}^{k,m} = \frac{1}{m^k}(1 + z + ... + z^{m-1})^k$$

The coefficients allow the iterative result to be calculated directly from the $k(m-1)+1$ values about $X(t)$ efficiently. They are interpreted as the probability mass function arising from a convolution of k discrete uniform distributions on the interval $[X(t)-\frac{m-1}{2}, X(t)+\frac{m-1}{2}]$. The result for $k > 1$ is a tapering window with finite support which approaches a gaussian distribution in the limit of k.

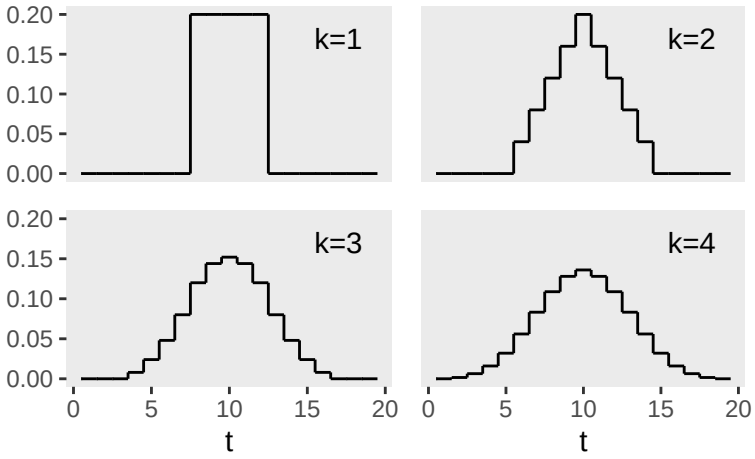

Figure 6: The filter produced by KZ(5,k) for k = 1,. . . ,4.

The KZ filter is classed as a "low-pass" filter in the field of signal analysis - high frequencies are suppressed in the filter output due to averaging of nearby readings, while low frequencies are allowed to pass to the filtered output. The proportion of energy passed from a given frequency can be determined with the energy transfer function provided by Yang and Zurbenko (2010). A "cut-off" frequency can also be determined for an arbitrary threshold level of attenuation; the cut-off frequency will be lower with increasing window size m and

iterations k. A formula for the commonly referenced "half-power point" is given by

$$\nu_c \approx \frac{\sqrt{6}}{\pi}\sqrt{\frac{1-.5^{\frac{1}{2k}}}{m^2-.5^{\frac{1}{2k}}}}$$

Selection of the two parameters are often determined heuristically along with knowledge of the underlying process, a desired cutoff point, and the noise apparent in the signal. Working with hourly temperature data strongly governed by diurnal and annual constituents, for example, a 5-day window ($m = 240$) and 1 or 2 iterations could be sufficient to suppress the daily cycle. Yet variance owing to exogenous factors (e.g. periods of cloud cover) and otherwise unexplained or random fluctuations may lead to selecting a larger window - perhaps 15 days - and perhaps 2 or 3 iterations to obtain the underlying annual component.

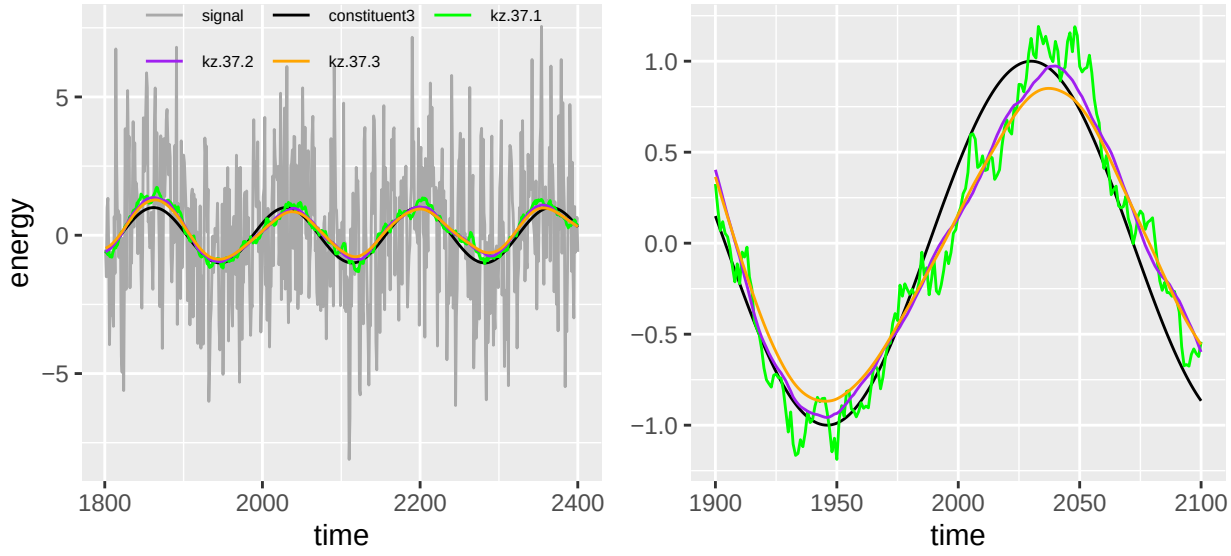

Figure 7: (left) Simulated signal, the f_{168} constituent, and output of KZ filter applied with m=37 and k=1,2,3; (right) zoomed in and with the raw signal removed.

Determination of optimal settings for these parameters may be approached as an (unsupervised) machine learning task - i.e. by partitioning or re-sampling the data so that a grid search can be performed in a sensible region of the parameter space pre-determined with guidance from the formulas derived from the energy transfer function. Kernel smoothing methods generally do not provide forecasting ability, so validation cannot be performed

out-of-time and some thought must be given to the in-sample design. A loss function could be constructed based on out-of-fold/out-of-bag reconstruction error and variance between smoothed results across (re-)samples.

The KZ filter has useful properties which make it well-suited to de-noising signals and separating waveforms embedded in noisy data. As a simple moving average it is robust to missing observations dispersed within the data (though longer runs of missingness become insurmountable as they approach and surpass the length of the window). The method can be re-applied to a signal minus the output of a previous application of the method in order to isolate and remove increasingly higher frequencies sequentially. At the most simple level it may be used for basic exploratory or graphical data analysis. In more advanced applications, it could be used as the basis of a filtering or convolution layer(s) in neural network architectures to effectively extract embedded features for use downstream in the network.[3] This paper's interest lies in its extension to the problem of estimating the spectral density of a signal, the focus to which we now turn.

Part II - Kolmogorov-Zurbenko Fourier Transform

We discussed the challenge of estimating spectra in the presence of noise with FFT. We also reviewed the KZ filter as an iterated sliding window moving average which suppresses the variance from certain sources - namely, the variance attributable to constituent waveforms located above a threshold frequency, to non-cyclical features of the data generating process, and to the stochastic variance of e.g. a white noise. The Kolmogorov-Zurbenko Fourier Transform (KZFT) extends the KZ filter into the frequency domain as a k-times iterated Sliding Window Fourier Transform with window size m applied at some frequency ν. A symmetric filter applied in the frequency domain determines a *range* of frequencies whose energy may pass through the filter while suppressing those which are more distant. KZFT is classed then as a *band-pass* filter because it is used to attenuate energy leakage

from distant frequencies, thereby improving estimates at target frequencies and regularizing spectral density estimates.

Definition

KZFT is defined in Zurbenko and Porter (1998) as a "linear fourier filter" of the form

$$F(t,\nu,M) = \frac{a}{2M+1}\sum_{s=-M}^{M} exp^{-is\nu}X(t+s)$$

which is applied K times iteratively to a signal of length $2MK+1$. This can be expressed completely (with minor updates for consistency of notation and definitions) accounting for the iterations as:

$$KZFT_{m,k,\nu}\Big(X(t)\Big) = F^{(k)}(t,\nu,m) = \sum_{s=-k\frac{m-1}{2}}^{k\frac{m-1}{2}} a_s^{m,k} exp^{-i2s\nu}X(t+s)$$

where $a_s^{m,k}$ are the same polynomial coefficient weights given previously. This provides an efficient estimator for power at the specified frequency ν. Zurbenko (1986) shows that as an estimator for spectral density, KZFT is asymptotically nearly optimal from the point of view of mean square error under differentiability conditions on the spectral density.[4] To analyze the complete spectrum within a window we apply the algorithm at each Fourier frequency from f_0 to $\frac{f_{m-1}}{2}$. Setting the window equal to the length of the original signal would produce a power spectrum similar to FFT in the first iteration. Subsequent iterations concentrate the weighting locally and further attenuate the contribution of outside frequencies. The 'sliding' of the window across the signal suggests that the power at a given frequency is estimated repeatedly - once at each index of the original signal - and the method produces estimates which can then vary over time. KZFT is robust to non-stationarity as a result and can be used to estimate energy dynamically over time.

We contrast the results of the classical periodogram produced by FFT (on energy scale)

to the KZ-periodogram obtained from applying a $KZFT_{4200,6}$ to our simulated signal with .05 SNR. Our signal is stationary by design so a spectrum can be sensibly estimated as the average result over the full signal. On visual inspection we see clearly that the estimated energies for the three constituent frequencies are nearer the true values. The noise does not appear reduced in the remainder of the spectrum, however.

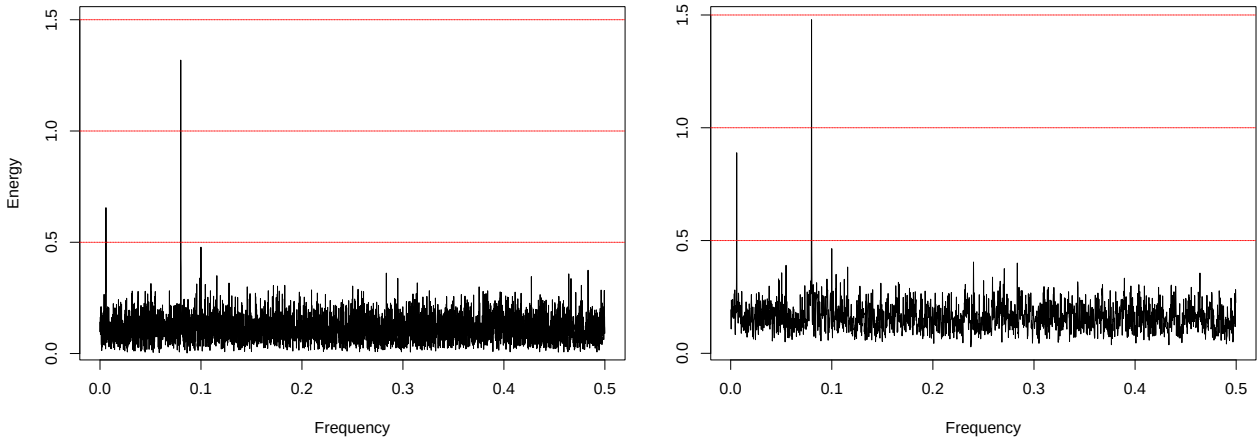

Figure 8: Spectral density (energy) based on FFT (left) and on $KZFT_{\nu,4200,6}$ to produce the KZ-periodogram (right)

Discussion

This definition or at least what it implies for implementation is worth a more detailed discussion. First, in contrast with the standard FFT, KZFT does not utilize all values in the signal but only those falling within the bandwidth of a convolved uniform kernel centered at $X(t)$. Let us begin in the middle of the signal at $X(0)$. At the first iteration, the DFT calculated at index 0 will include the contribution of nearby values $X(t);\ t := -\frac{m-1}{2}, ..., 0, ..., \frac{m-1}{2}$. The second iteration takes the result from the first over the same indices - so m outputs from the first pass are needed. Then the original samples contributing in the second pass are $X(t);\ t := -\frac{2(m-1)}{2}, ..., -\frac{m-1}{2}, ..., 0, ..., \frac{m-1}{2}, ..., \frac{2(m-1)}{2}$. For each output value $F^{(k)}(t)$, the $k(m-1)+1$ values centered about t from the original signal contribute subject to the attenuation produced by the weights of the tapering window. Practically this means that we must choose from among several options how to handle cases where the window

extends beyond the tails of the signal:

1. accept an output length reduced by $k(m-1)$ (half reduction at each end), in essence discarding these estimates;
2. pad the original series with zeroes so that the filter may be centered at each value of the input signal but suppressing (biasing toward 0) the power estimates near the tails;
3. allow for an asymmetrically shrinking window, down to a "half+1" window at the extremes (increasing the variance of estimates).

For a long series and/or short window this may have no practical consequence for estimates. Increasing either the window size or the number of iterations will of course increase the extent of window loss at each end of the signal, which can meaningfully increase the variance of estimates when dealing with short (relative to the desired window length) time series.

KZFT is naturally robust to non-stationarity, though this will also depend on the window size and time frame over which the non-stationarity is detectable. A window which is far larger than necessary if interested only in higher frequencies will permit a non-stationary component in the process to impact estimates if it is not "nearly" stationary within the window. In this case, a smaller window will resolve the issue while preserving the ability to detect the higher frequencies of interest. If non-cyclical trends or breaks/state changes are apparent within a smaller window then it may be necessary to de-trend or otherwise adjust the original signal prior to analysis, if possible.

Finally, available implementations are based on repeated direct calculations of the DFT - i.e. not utilizing the FFT algorithm. This allows for missing observations but at added computational expense. Within a window m the algorithm is $O(m)$ complex at a single observation $X(t)$ for a single frequency ν and for one iteration. If we evaluate all fourier frequencies within the window (estimating the power spectrum at one location) then this becomes $O(m^2)$ complex - equivalent to the DFT of a signal length m. Repeating at every value of the input signal (a sliding window) produces $O(nm^2)$ complexity *per iteration* and

ultimately $O(knm^2)$ complexity in total. In other words, the computational cost scales poorly with window size. Consider a series of length 8,760 representing hourly readings taken over a year's time. Producing the KZ-periodogram with a window of length 169 (about a week) requires about 250.2 million operations per iteration - not terrible for modern hardware - but with a window length 4,369 (about 6 months) the overhead grows to 167.2 billion per iteration, or about 502 billion in total if $k = 3$ iterations is desired. The prohibitive cost may demand some creativity by the user needing to analyze the lower end of a spectrum. Some options are:

1. Reduce the resolution of the signal by reducing the sampling frequency - i.e. reducing from an hourly sample rate to daily or weekly by aggregating the original readings;
2. Instead of evaluating all frequencies within a window, evaluate only a specified range;
3. Parallelize the algorithm over multiple threads to reduce clock-time in proportion with the number of available cores;
4. If data is complete or missing values can be imputed beforehand then the algorithm can be implemented using FFT, providing improved performance of $O(knm \log m)$.

Currently available implementations for computing the KZ-periodogram can produce impossibly long run-times with long signals and a large window. An implementation along the lines of option 4 above is sorely needed to expand the range of applications where KZFT can be practically applied. For now, a small modification can be made to the existing implementation of the `kza` R library to realize option 3, which can yield quite a large reduction in clock time (by a factor of $\frac{1}{\text{number of cores}}$) needed to evaluate a wider spectrum. A revised implementation of this kind is provided in Appendix 1 and was used to obtain results herein.

Part III: Spectral Analysis Traffic Accidents in the Capital District

Study Data

Our research question is whether the occurrence of traffic accidents around the capital region of New York over time exhibits cyclicality. The basis for our study is a dataset produced by Moosavi, et al (2019) compiling US Traffic Accident events broadcast by various entities including "US and state departments of transportation, law enforcement agencies, traffic cameras, and traffic sensors within the road-networks." These events are sourced from two traffic event API services, integrated (with likely duplicate events removed), geocoded, and published for research use.

Table 1: Two source event records which overlap in time

ID	Start_Time	End_Time	Zipcode	Description
A-2149438	2019-04-05 07:23:24	2019-04-05 08:23:08	12205	Right hand shoulder closed due
A-2149419	2019-04-05 08:17:01	2019-04-05 09:31:47	12204	Right lane blocked due to acci

We subset the full data to those events occurring in selected capital region zip codes in 2019. There are `n_events` accident events recording in the time period for the area of study. Each event comes encoded with a start time and end time. We discretize these event records into hourly periods with the following logic:

1. Create a sequence of hourly periods spanning the time period covered by the research data;
2. "Overlap-left-join" the events to the hourly series based on `Start Time` and `End Time` of the event and for the hourly periods - each event is matched to the one or more periods it falls in;
3. Measure the portion of each hour $\in$ `(0, 1.0]` which is covered by the event;

4. Summarize by hourly period the total duration of all events (any period with no events records 0 energy), denoting the resulting series as our signal `x(t)`.

Table 2: Discretization of the two events in Table 1 to create signal X_t

t	t_start	t_end	x
26575	2019-04-05 06:00:00	2019-04-05 07:00:00	0.0000000
26576	2019-04-05 07:00:00	2019-04-05 08:00:00	0.6100000
26577	2019-04-05 08:00:00	2019-04-05 09:00:00	1.1019444
26578	2019-04-05 09:00:00	2019-04-05 10:00:00	0.5297222
26579	2019-04-05 10:00:00	2019-04-05 11:00:00	0.0000000

An event lasting exactly one hour contributes 1.0 energy to the series in total; this contribution is fully allocated to the 12th hour of the given day if it begins at exactly 11:00:00 and ends at 12:00:00 EST. If the event begins at 11:15:00 and ends at 12:15:00 then 0.75 energy is allocated to hour 12 and 0.25 to hour 13.[5] The theoretical range of `x(t)` is $[0, \infty)$, though at the high end this would be a very bad day for drivers in Manhattan. The maximum energy observed in any hour is 5.9561.

Figure 9 shows the smoothed empirical distribution of hourly energies, with and without the point mass at 0 (most hours having no events). Figure 10 displays a two-week period in the signal. We see here anecdotal evidence of at least two unsurprising patterns: energy is highest on weekdays and lowest on weekends; and energy appears to concentrate around the 8th and 17th hours.

Analysis

We estimate the power spectral density of the signal to detect periodic components which may be embedded in it. In choosing appropriate parameters we anticipate waveforms with likely periods ranging from hours up to potentially monthly. We also recognize that traffic accidents are inherently random outcomes - "hourly traffic events" therefore presents

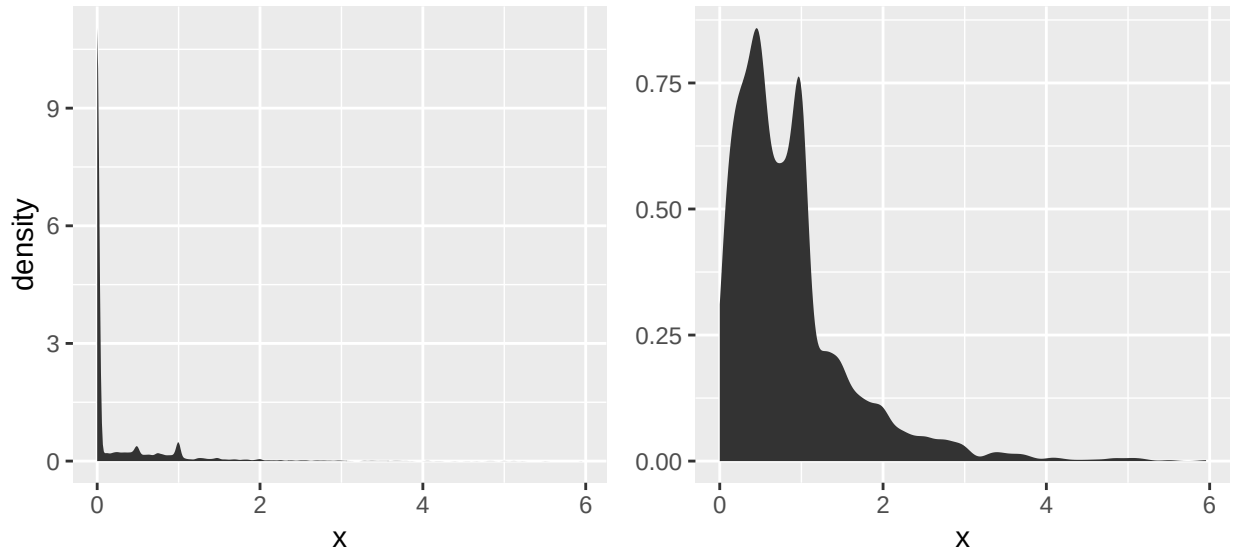

Figure 9: Smoothed distribution of measured hourly energy is shown in total (left) and with the point-mass at 0 removed (right).

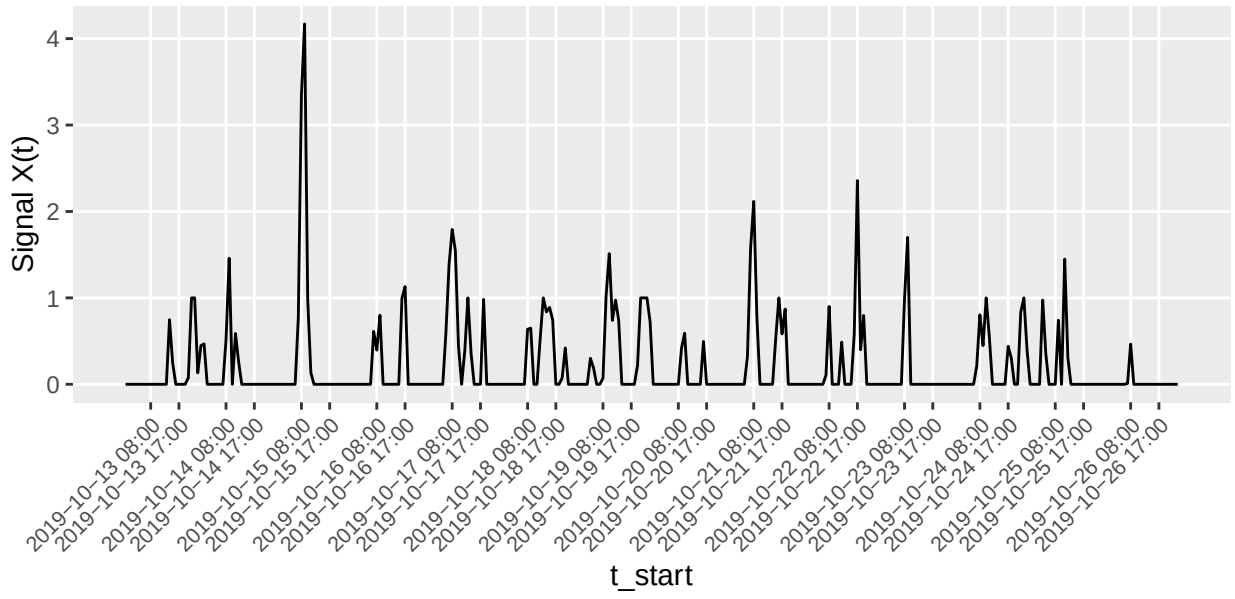

Figure 10: Signal energy for two-week period from Sunday, October 13 through Saturday, October 26, 2019

a highly noisy signal and we will benefit from the smoothing produced by repeated iterations of KZFT. We select a window size $m = 2184$ (13 weeks) and $k = 3$ iterations given these considerations.

The periodogram (Figure 11) reveals four dominant constituents at frequencies .04167 (24 hours), .0833 (12 hours), .1250 (8 hours), and .005952381 (168 hours or 1 week), along with a number of others showing intermediate to minor importance or energy. The ten highest powered frequencies are all higher-order harmonics of the fundamental weekly frequency, indicating the spectrum is largely approximating a non-sinusoidal weekly pattern. A number of others (62.4, 19.675676, 12.066298 day cycles) are harmonics of a frequency with 364-day period which itself is not visible within the selected window. In other words, the signal is strongly determined by a weekly and possible annual pattern which are not well-specified as a linear combination of sinusoids but can at least be approximated as such.

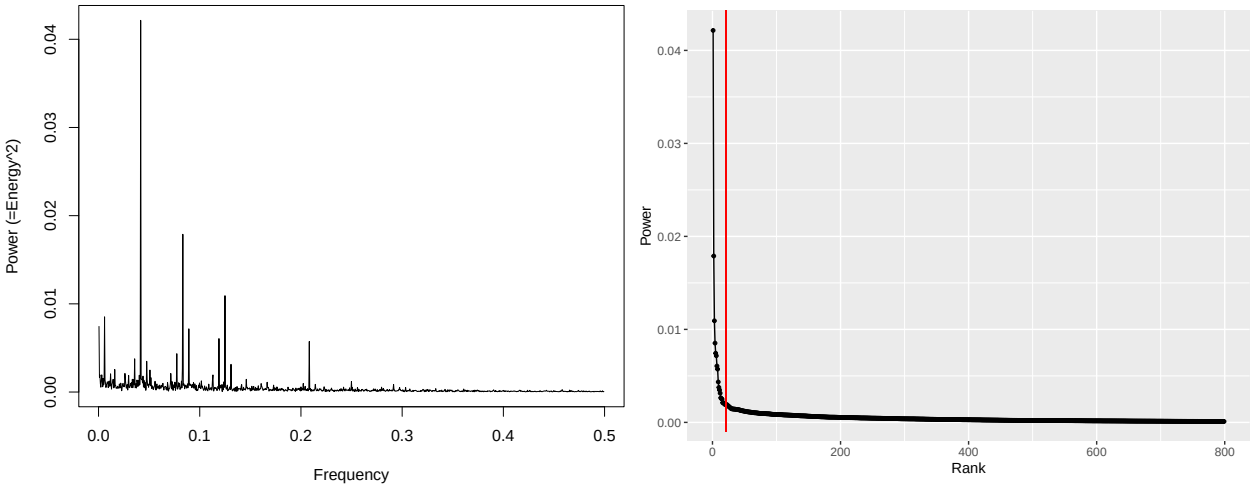

Figure 11: (left) Power Spectral Density of hourly traffic accident signal; (right) scree plot of power spectrum constituents

Signal Reconstruction

Signal reconstruction is possible by summation of individual KZFTs estimated about the dominant constituent frequencies detected in the original signal. De-noising signals in this way is a typical use for spectral estimates when working with signals produced by physical

phenomena - sounds, vibrations, ocean tides and such. In our context, we may think of the signal reconstruction in terms of producing a risk pattern for open or ongoing traffic incidents over time.

A scree plot provides visual indication of the importance of including each incremental constituent. Figure 12 shows the five dominant constituents individually over a week in June 2019 and the reconstructed signal using these components over the same two weeks. We observe in the reconstructed signal a pattern of twice-daily peaks occurring at 8:00 am and 5:00 pm, with higher peaks Monday-Friday compared to Saturday and Sunday. This demonstrates that a non-trivial calendar-week pattern can be approximated with just a small number of sinusoidal curves.

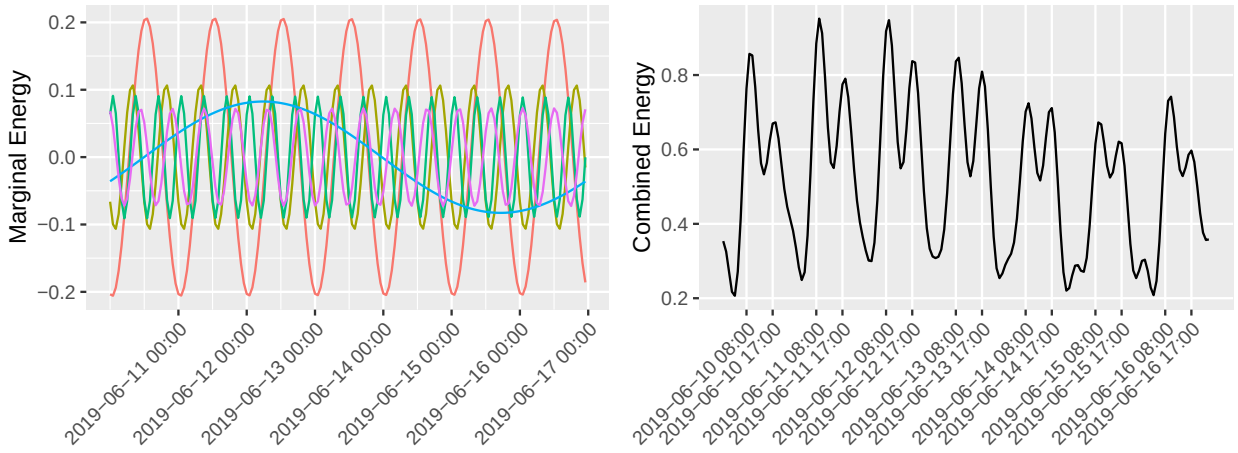

Figure 12: (left) Top 5 constituent waveforms shown individually; (right) Relative event risk based on these

From the estimated PSD and the scree plot it appears we can complicate the signal reconstruction and explain greater systematic variation by including more than 5 components. We select the top 15 frequencies by rank which are also harmonic constituents of the fundamental weekly frequency. In pure rank terms these all fall within the top 20 frequencies ranked by power estimate.

The resulting expected risk function reveals further interesting and intuitive temporal patterns. Weekdays exhibit the twice daily peaks at 08:00 and 17:00 mentioned earlier, but we now observe a single peak on Saturdays and Sundays occurring about mid-day, between

the 12:00 and 13:00 hours. We also see increased energy in the late evening/early morning hours of Fridays and Saturdays, though also and less intuitively, Wednesdays. This can indicate increased roadway traffic in those hours compared to weeknights, higher incidence of impaired drivers, poor approximation of the true pattern, or some combination of these. Finally, inclusion of the additional components has clearly increased the variance of the reconstructed signal, which now exhibits higher peak levels and faster ramping to and from these peaks.

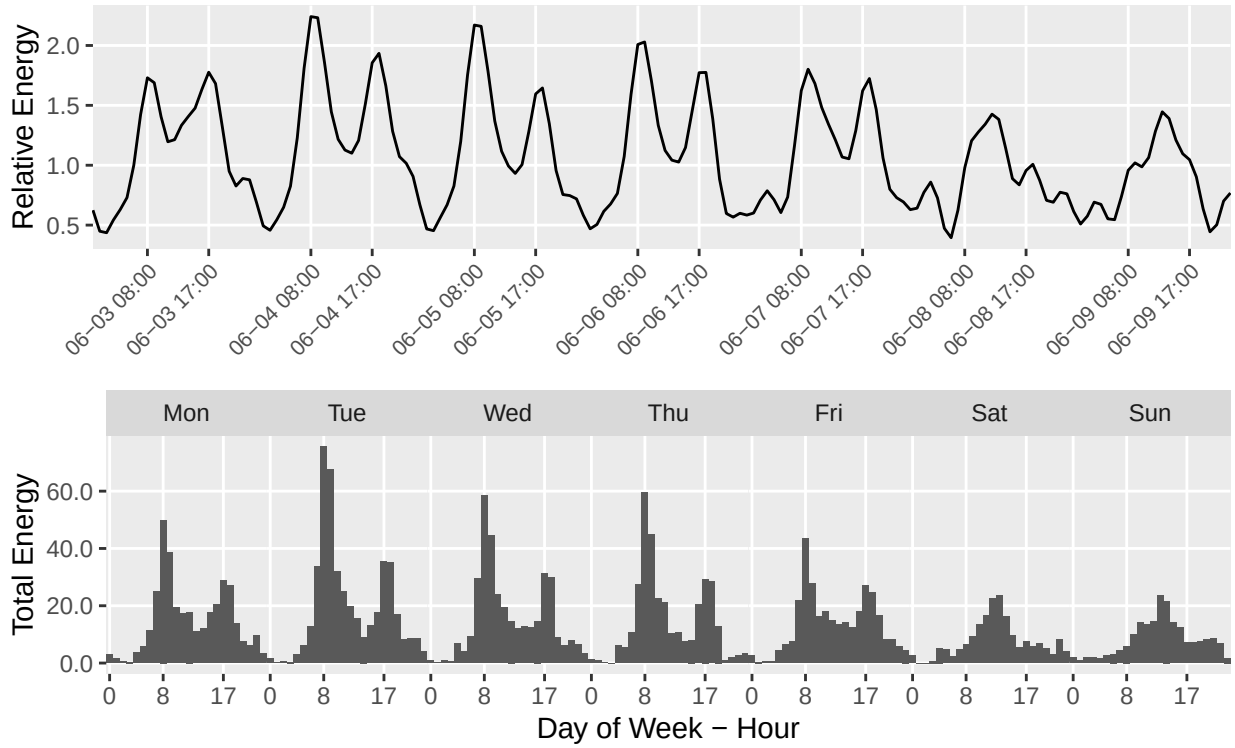

Figure 13: (top) Signal reconstruction based on 14 constituent frequencies and f_0; (bottom) Average hourly energy by weekday-hour for 2019

Because the signal is strongly determined by a fundamental weekly frequency and its harmonics, we could be justified in ultimately selecting some method based in the time domain to produce estimates. We may compare the results above to a naive estimator for day-hour periods - i.e. fitting a regression to these contrasts or simply calculating the mean by group. We note though that such a model would utilize 168 (24×7) degrees of freedom, ignoring order and making no assumptions about smoothness in time.[6]

Discussion

etc

Notes

[1]The average unit cost of an IoT sensor fell from $1.30 in 2004 to $0.44 in 2019 according to Microsoft's 2019 Manufacturing Trends Report

[2]Fourier first wrote down his equations in an 1807 manuscript, and later in his 1822 published work. But as is often the case it appears Carl Friedrich Gauss was first to discover the method in 1805 along with an efficient "divide and conquer" algorithm for its computation which closely resembles the FFT algorithm developed 160 years later.

[3]Chapter 10 of Žurbenko et al. (2021) demonstrates application of the KZ filter to image reconstruction and object boundary detection problems. While weights in a filtering layer are typically learned during network training, fixed weights are sometimes used with positive result where domain knowledge permits. See for example Dapello, et al (2020).

[4]See Theorems 4.6 - 4.8, p.p. 96-97, and formulas 2.37-3.48, pp 123, Thm 2.6 pp 124 MSE establish satisfies theoretical optimal conditions

[5]Our decision to allocate energy to the periods spanned by an event suggests applications in areas such as transportation logistics and resource planning. Alternatively, we could allocate the full energy of an event to the period of its initiation which would be more in line with a risk perspective.

[6]Our signal can be split into about 4 independent blocks, though we do not treat them independently. It would seem that $4 \times 2 \times 15$ would serve as a good estimate for total d.f. for a reconstruction based on 15 constituents derived from a single iteration of KZFT, but this would decrease with additional iterations - Formula 1.1.19 Loneck?

Bibliography

Dapello, Joel, Tiago Marques, Marcus Schermer, Haofeng Li, David Cox, and James J. DiCarlo. 2020. "Simulating a Primary Visual Cortex at the Front of CNNs Improves Robustness to Image Perturbations." Advances in Neural Information Processing Systems 33 (NeurIPS 2020): 11467-11478.

Dominguez, A. "Highlights in the History of the Fourier Transform." *IEEE Pulse*, Volume 7, Issue 1, January 2016. https://ieeexplore.ieee.org/stamp/stamp.jsp?arnumber=7389485.

Gundla M.P (2025). The Transformative Impact of IoT on the Insurance Industry, European Journal of Computer Science and Information Technology, 13 (5), 77-90

F. J. Harris, "On the use of windows for harmonic analysis with the discrete Fourier transform," Proceedings of the IEEE, vol. 66, no. 1, pp. 51-83, Jan. 1978, doi: 10.1109/PROC.1978.10837.

McCallum, J. (2023). U.S. Bureau of Labor Statistics (2026) – with minor processing by Our World in Data. https://ourworldindata.org/grapher/historical-cost-of-computer-memory-and-storage

Microsoft. (2019). 2019 Manufacturing Trends Report. Microsoft Industry Clouds. https://info.microsoft.com/rs/157-GQE-382/images/EN-US-CNTNT-Report-2019-Manufacturing-Trends.pdf

Moosavi, S., et al. "A Countrywide Traffic Accident Dataset." arXiv:1906.05409 (2019). Data accessed from https://www.kaggle.com/datasets/sobhanmoosavi/us-accidents/data

Yang, W. and Žurbenko, I. "Kolmogorov-Zurbenko Filters." *WIREs Computational Statistics*. (2010) 2, no. 3, 340–351, https://doi.org/10.1002/wics.71, 2-s2.0-78651530981.

Žurbenko I. (1986). *The Spectral Analysis of Time Series*. New York: North-Holland.

Žurbenko, I. and Porter, P. S. (1998). Construction of high-resolution wavelets. Signal Processing, 65(2), 315–327. https://doi.org/10.1016/s0165-1684(97)00226-0.

Žurbenko, I., et al. (2021). *High-Resolution Noisy Signal and Image Processing*. Newcastle upon Tyne: Campridge Scholars Publishing.

Appendix 1: parallelized `kzp()` (Unix/Linux/MacOS)

We provide a modified implementation of the `kzp()` method from library `kza`. This version provides parallelization over the frequency bins - i.e. the energy estimates at each frequency within the window are produced concurrently across multiple cores and combined in the result. This can provide a significant reduction in processing time (see additional notes below). The `parallel` package is required. An implementation for Windows is provided in Appendix 2.

The method has also been modified to optionally produce estimates for a smaller range of frequencies within a window, specified in the `f_range` argument. This preserves frequency resolution without requiring processing of every frequency bin. The user may supply a length 2 vector specifying the minimum and maximum frequency bin to evaluate; `f_range=c(0,m-1)` is the default and original behavior.

Lastly, the method has been modified to return the estimated energy and power normalized for window size `m` and iterations `k`, in addition to the original, un-normalized result. Each is accessible in the return object and selectable for plotting by `plot.kzp()` (see Appendix 3).

You can determine the number of cores available on your machine prior to calling the method. It is advisable to always use 1 or 2 fewer than total available cores, so that resources will be left for other processes and the system remains responsive.

```
## determine how many cores available and how many to use
library(parallel)
n_cores <- detectCores() - 2
```

The user allocates the desired number of cores in the method call (and optionally may also supply a frequency range).

```
kzp <- function(y, m=length(y), k=1, f_range = NULL, ncores = n_cores)
{
  M<-(m-1)*k+1
  n=length(y)

  ## default to full spectrum as determined by m
  ## but allow user-specified range (provided in frequency-bin)
  if (is.null(f_range)) {
    f_est <- c(0,m-1)
  } else {
    f_est <- f_range
  }

  ## parallelized over the frequencies to be evaluate
  z_tmp <- parallel::mclapply(
    X = f_est[1]:f_est[2],
    FUN = function(i, x, m, k) kzft(x,f=i/m,m,k=k),
    x = y, m = m, k=k,
    mc.cores = ncores
  )

  z <- matrix(unlist(z_tmp), nrow = n, ncol = f_est[2]-f_est[1]+1)

  d <- apply(z, 2, function(z) {(abs(z)^2)*M})
  a <- colMeans(d, na.rm=TRUE)

  if(is.null(f_range)) {
    a <- a[1:(m/2)]
    f_range <- c(0, length(a)-1)
  } else {
    a <- a[1:(f_range[2] - f_range[1] + 1)]
  }
  b <- 2*sqrt(a/M)
  c <- b^2

  structure(list(
    periodogram = a, ## original
    energy = b, ## estimated energy
    power = c, ## estimated power
    window=m,
    f_range=f_range, ## for user-specified frequency ranges
    k=k,
    var=var(y),
    smooth_periodogram=NULL,
```

```
    smooth_method=NULL,
    call=match.call()
  ),
  class = "kzp")
}
```

This implementation is based on `parallel::mclapply()` which relies on forking and therefore only works on Unix-like systems (Linux, Apple OS). Unlike Unix systems, Windows OS do not support POSIX system forking, an efficient method which allows child processes to access objects in shared memory. The method will not throw an error if called on Windows but will simply process with a single core - the Windows user must be aware they will not obtain the desired parallelization and will not be alerted to this.

Concurrently evaluating frequencies in the spectrum of a window can provide significant reduction of processing time (wall clock) compared to the sequential, single-threaded implementation currently available. The reduction in run time is in proportion to the number of CPU cores allocated; e.g. allocating 10 cores (available on many higher-end laptops) will reduce the run time by 90%. A more powerful server may contain upwards of 300 cores, allowing an extreme reduction of processing time (if you can grab them all without annoying your neighbors). This is critical to enabling analysis of long signals (i.e. many thousands of readings) with a large window. Tidal and atmospheric data, for example, are available today at 6-minute resolution going back over a decade; the lunar nodal cycle is ~18.6 years long.

Appendix 2: parallelized `kzp()` (Windows)

A similar implementation can be realized with `parallel::parLapply()` which relies on PSOCK clusters supported by Windows. A cluster of child processes is created and objects in memory are copied to each, making it less efficient than forking. The user must be mindful of their total system memory and the size of required data objects (i.e. the signal object) at the risk of hanging their R session or crashing the system. Only distribute objects needed by `kzft()`.

```
cl <- makeCluster(n_cores, type = "PSOCK")
clusterExport(cl, varlist = c("signal_object", "kzft"))
```

The implementation of `kzp()` provided above would be re-written slightly replacing the command at line 15 with:

```
  z_tmp <- parallel::parLapply(
    cl = cl,
    X = f_est[1]:f_est[2],
    FUN = function(i, x, m, k) kzft(x,f=i/m,m,k=k),
    x = y, m = m, k=k
  )
```

After calling `kzp()`, you must stop the cluster to free the allocated cores for regular system use.

```
stopCluster(cl)
```

Appendix 3: Enhanced `plot.kzp()`

The `plot.kzp()` method from `kza` is rewritten to accommodate several options for scale of the plotted spectral density with the `scale` argument. The original option is preserved - this displays on an information scale. Options for power and energy spectral densities are added. We have also added the option to include or exclude f_0 (the DC component) from the plot - including it will often conceal what is happening across the spectrum due to scale differences.

Support for plotting smoothed spectral densities is removed out of convenience rather than necessity.

```
plot.kzp <- function(x, remove0 = TRUE, scale = NA, ylab = '', ...) {
  if (!is.na(scale)) {
    stopifnot("scale must be one of 'original', 'power', or 'energy'"
              = scale %in% c("original", "power", "energy"))
  }

  if(is.na(scale) | scale == "original") {
    p <- x$periodogram
  } else if (scale == "energy") {
    p <- x$energy
  } else if (scale == "power") {
    p <- x$power
  }

  if(x$f_range[1] == 0 & remove0) {
    dz <- p[-1]
    omega <- ((x$f_range[1]+1):x$f_range[2])/x$window
    plot(omega, dz, type="l", xlab="Frequency", ylab=ylab)
  } else  {
    dz <- p
    omega <- (x$f_range[1]:x$f_range[2])/x$window
    plot(omega, dz, type="l", xlab="Frequency", ylab=ylab)
  }
}
```

Appendix 4: `kzft()`

The `kzft()` method as provided in R library `kza` (with some minor formatting changes) is provided here for completeness and posterity. The package is no longer maintained and will be archived by CRAN once it begins to fail automated build checks. Note that this method is not exposed upon loading the library as is expected of R packages; the user must source the method into their environment from file `kzft.R` which is bundled in the package or can be obtained from the CRAN GitHub mirror.

```
kzft <- function(x, f=0, m=1, k=5)
{
  h <- floor((m-1)/2)
  t  <- ceiling((m-1)/2)
  x <- append(x, rep(NA,t*k), after = length(x) )
  x <- append(x, rep(NA,h*k), after = 0)
  z <- as.complex(x)

  for(i in 1:k) {
    s     <- rep(0, length(z))
    count <- rep(0, length(z))
    count <- count + !is.na(z)
    z[is.na(z)] <- 0
    s <- as.complex(s) + z

    for(j in 1:h) {
      # add offset
      z   <- c(rep(NA, j), x[1:(length(x)-j)])
      z <- z * exp(complex(real = 0, imaginary = 2*pi*f*j))
      count <- count + !is.na(z)
      z[is.na(z)] <- 0
      s <- s + z
    }
    for(j in 1:t) {
      z   <- c(x[(j+1):length(x)], rep(NA, j))
      z <- z * exp(complex(real = 0, imaginary = -2*pi*f*j))
      count <- count + !is.na(z)
      z[is.na(z)] <- 0
      s <- s + z
    }
```

```
    z <- s/count
    x<-z
  }
  z <- z[(t*k+1):(length(z)-(h*k))]
  return(z);
}
```